



Universidad  
Indoamérica

UNIT 4 · FUNDAMENTALS OF OPERATING SYSTEMS

---

# Processes and memory

Universidad Tecnológica Indoamérica    Lab TD3

# The process: a running program and its context

---

- Context: memory, registers, state of the descriptors...
- The OS runs many processes at once (unit 1, A1.4)
- Each one has its own **private address space**: isolation, security and the virtual memory abstraction
- It hides the kernel and the other processes

# Virtual addressing

- An  $n$ -bit address allows  $2^n$  addresses
- Address space = the addresses + the state of every cell
- Every process sees **only its own**

## On x86-64 today

64-bit pointers, but only **48** bits are used: 256 TiB of virtual space, the lower half for the process and the upper half for the kernel. With 5-level paging, 57 bits (128 PiB).

# The zones of a process: stack, heap, data, code

## Real output

```
$ ./zones
zone          address      perm  region
code   (main)  0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)  0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)  0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)    0x7fff6c1e68b4  rw-p  [stack]
```

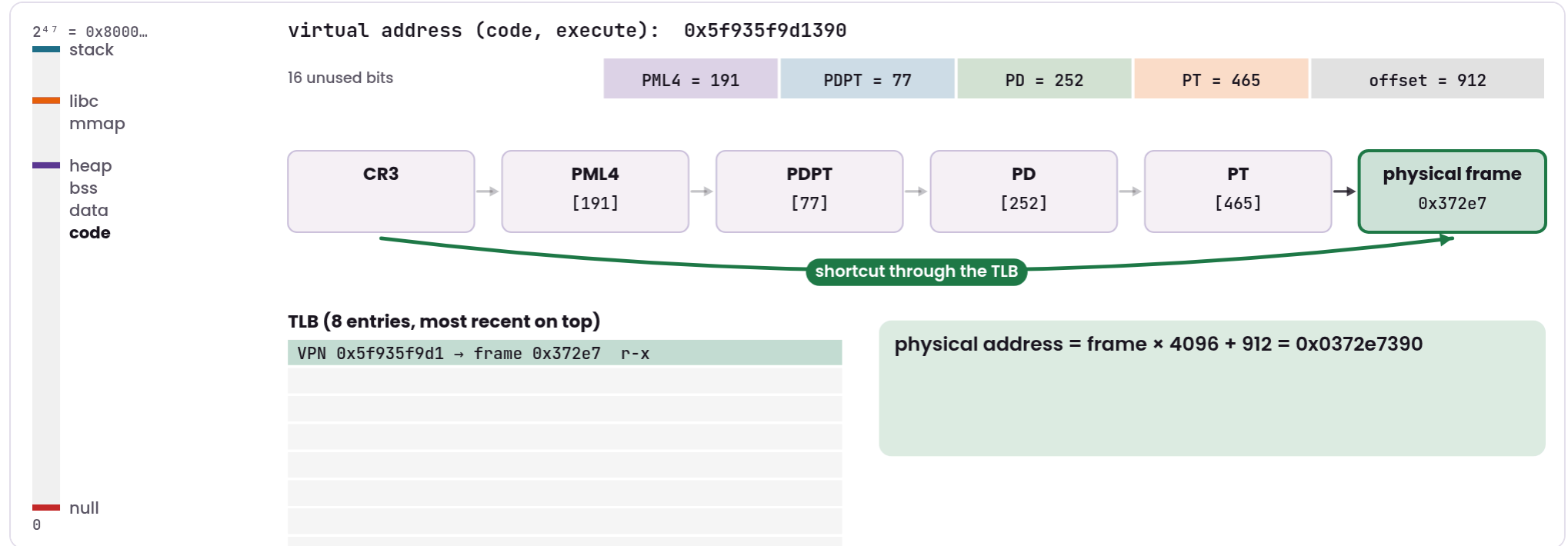
**Stack**: local variables · **heap**: small `malloc` s · **mmap**: large `malloc` s and libraries · **data/bss**: globals and `static` · **code** (text): the instructions, read-only and executable.

## From virtual to physical: paging

- The CPU does the translation: the **MMU**, with the **TLB** cache
- Memory is managed in **pages**: 4 KiB on x86-64 (and huge pages of 2 MiB or 1 GiB); on ARM also 16 or 64 KiB
- Each virtual page points to a physical **frame**, through a table (on x86-64, 4 levels)
- The address is split into fields: table indices and offset

**Fix to the original:** it said "blocks of 2 KB (4 KB or 2 MB on x86)". On x86 the base page is 4 KiB; 2 KB is not an x86 page size.

# Translating real addresses



## Lazy allocation: the page exists when it is touched

### Real output

```
$ ./lazy 1024
start                               VmRSS =    1756 KiB   page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                         VmRSS =    1952 KiB   page faults so far = 8
after touching half of the pages    VmRSS =   526300 KiB  page faults so far = 131081
pages touched: 131072 (4096 bytes each)  page faults: 131073   1968 ns per fault
after free                           VmRSS =    2012 KiB   page faults so far = 131081
```

- Much more efficient than allocating everything up front
- What if there are no free frames left? A rarely used page goes to disk (swap) or is compressed in RAM (zswap, zram): page replacement →

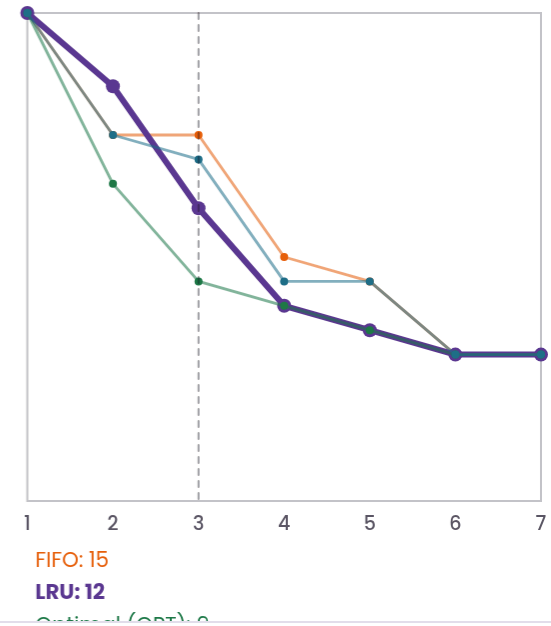
# No free frames: which page goes?

## LRU · 3 frames

| ref.    | 7        | 0        | 1        | 2        | 0 | 3        | 0 | 4        | 2        | 3        | 0        | 3 | 2 | 1        | 2 | 0        | 1 | 7        | 0 | 1 |
|---------|----------|----------|----------|----------|---|----------|---|----------|----------|----------|----------|---|---|----------|---|----------|---|----------|---|---|
| frame 0 | 7        | 7        | 7        | 2        | 2 | 2        | 2 | 4        | 4        | 4        | 0        | 0 | 0 | 1        | 1 | 1        | 1 | 1        | 1 | 1 |
| frame 1 |          | 0        | 0        | 0        | 0 | 0        | 0 | 0        | 0        | 3        | 3        | 3 | 3 | 3        | 3 | 0        | 0 | 0        | 0 | 0 |
| frame 2 |          |          | 1        | 1        | 1 | 3        | 3 | 3        | 2        | 2        | 2        | 2 | 2 | 2        | 2 | 2        | 2 | 7        | 7 | 7 |
|         | <b>F</b> | <b>F</b> | <b>F</b> | <b>F</b> |   | <b>F</b> |   | <b>F</b> | <b>F</b> | <b>F</b> | <b>F</b> |   |   | <b>F</b> |   | <b>F</b> |   | <b>F</b> |   |   |
|         |          |          |          | -7       |   | -1       |   | -2       | -3       | -0       | -4       |   |   | -0       |   | -3       |   | -2       |   |   |

F = fault · -x = victim

## faults vs frames





## The memory interface: `brk`, `malloc`, `free`

C

```
int brk(void *end);           /* end of the heap */
void *sbrk(intptr_t delta);
void *malloc(size_t size);    /* NULL on failure */
void *calloc(size_t n, size_t size);
void *realloc(void *p, size_t size);
void free(void *p);           /* free(NULL): no-op */
```

Do not use `brk` / `sbrk` directly: use `malloc`.

- glibc: small blocks on the heap (`brk`); from 128 KiB, a `mmap` of their own (visible in `zones`)
- Linux is optimistic (*overcommit*): `malloc` almost never returns NULL; if memory runs out, the **OOM killer** terminates processes
- Double `free` or `free` of something not allocated: undefined behaviour
- Debugging: GDB, Valgrind and today `-fsanitize=address` (ASan)

# The process for the OS: the PCB

## Static

PID, PPID, owner, access rights

## Dynamic

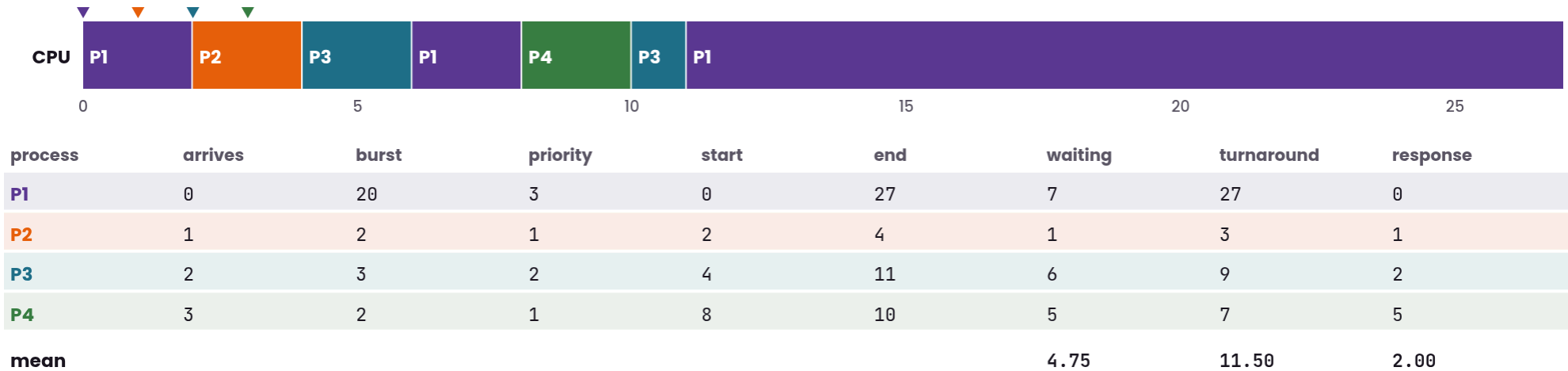
State, registers and program counter, priority, CPU times (user/system), open descriptors, limits ( `getrlimit` ), signals, page table, threads

## In Linux

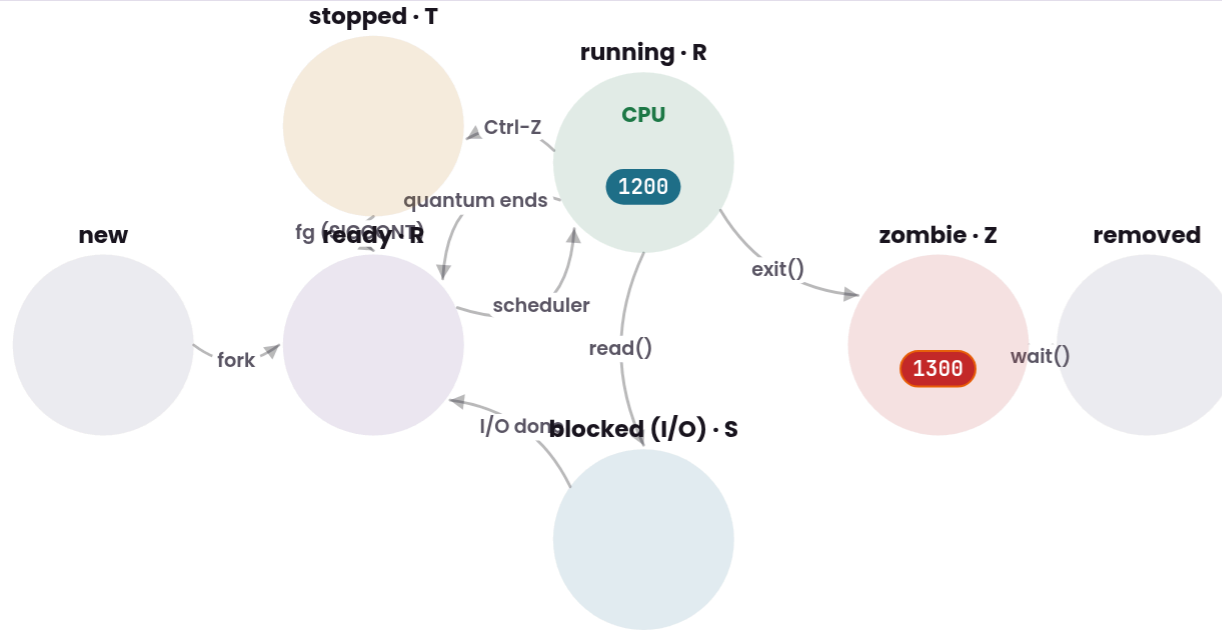
The PCB is `struct task_struct` (several KiB). It is inspected in `/proc/PID/`: `status`, `maps`, `fd/`, `limits` ... That is how `ps` and `top` work.

# Context switching and scheduling

Round Robin · quantum 2



# Process states



```
$ ps -o pid,ppid,stat,comm
  PID   PPID  STAT COMMAND
  1200   1100  R+   bash
  1300   1200  Z    lifecycle <defunct>
```

## events

- bash (1200) forks → 1300 is born
- 1200's I/O finished (interrupt) → ready
- 1300 exits with status 7: it stays a zombie until its parent calls wait

# How is a process created?

## Windows

`CreateProcess()` creates the process and loads the program in one step, with many options.

## Unix

1. `fork()` : a clone of the current process
2. The clone adjusts whatever it wants (descriptors, directory...)
3. `exec()` : it turns into another program

Four families: **create** ( `fork` ), **execute** ( `exec...` ), **terminate and signal** ( `exit` , `kill` → U6), **wait** ( `wait...` ).

Today also: `posix_spawn()` (fork + exec in one call), `vfork()` and `clone()` / `clone3()` , the real call behind `fork` and threads (A4.7).

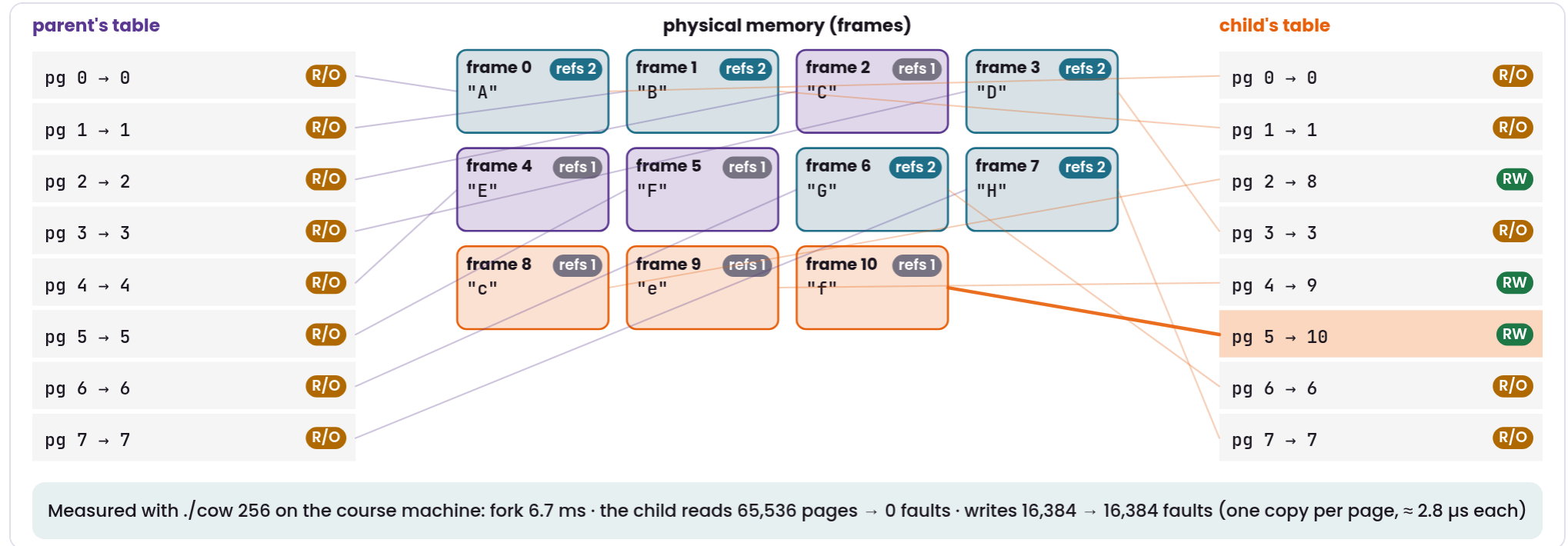
## Cloning a process: `fork` and its cost

- The child runs the same program with a **copy** of the parent's address space
- Creating the PCB is cheap; copying every page would be very expensive
- **Copy-on-write**: they are shared and copied only on write

### Real output

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096
bytes)
fork took 6.70 ms (only the page tables are
copied)
child reads 65536 pages: 0 page faults
(shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults
(one copy per page), 2753 ns per copy
parent: the child exited with status 0; my
copy still says 'x'
```

# Copy-on-write, page by page



## pid\_t fork(void)

- Returns **0 in the child**, the **child's PID in the parent** and -1 on failure
- Identical except for PID and PPID ( `getpid`, `getppid` )
- The child inherits signal handlers and mask, **not** pending signals
- It also inherits... the pending stdio buffer

C tds/td3/fork\_pid.c

```
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    printf("child %d, parent %d\n", getpid(), getppid());
    exit(EXIT_SUCCESS);
}

/* Parent: prints nothing after fork, but must wait for its child. */
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
```

### Real output

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```



# stdio buffers and fork

```
printf("My pid is %d / ", getpid());
pid = fork();
/* child */ printf("child %d, parent %d\n", ...); exit(0);
/* parent */ waitpid(pid, ...); return 0;
```

the parent leaves with `exit()`: it flushes its buffer

parent · stdout: fully buffered ended

(empty)

child · stdout: fully buffered ended

(empty)

stderr: unbuffered

system calls

```
[hijo] write(1, "My pid is 356446 / child 356448, parent 356446\n", 47)
write(1, "My pid is 356446 / ", 19)
```

pipe → cat → terminal

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

real output (run.sh):

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

## Running another program: `execve`

C

```
int execve(const char *path, char *const argv[], char *const envp[]);  
/* execl, execlp, execl, execv, execvp, execvpe:  
   l = list, v = vector, p = search PATH, e = environment */
```

- `argv`: `main`'s arguments; `envp`: the environment  
( `PATH`, `HOME`, `LANG`, `LD_LIBRARY_PATH` ...;  
`getenv` / `setenv` )

- If it works, it **does not return**: it replaces code, data, heap and stack
- It keeps PID, PPID and the open descriptors (except `O_CLOEXEC` ones)
- With the SUID bit, the effective UID becomes the file owner's ( `passwd`, `sudo` )
- If it fails, it returns `-1`

# fork + dup2 + exec + wait : the shell at work

**launch · PID 356524**

running

**ls -l demo · PID 356526**

reaped by wait

## child's descriptors

```
0 → terminal
1 → out.txt
2 → terminal
3 → -
```

## strace -f (real trace, summarized)

```
356524 execve("./launch", ["../launch", "out.txt", "ls", "-l", "demo"], ...) = 0
356524 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
356524 write(3, "ls", ...) ...
356524 clone(..., flags=...|SIGCHLD) = 356526
356524 close(3) = 0
356524 wait4(356526, <unfinished ...>
356526 dup2(3, 1) = 1
356526 close(3) = 0
356526 execve("/usr/bin/ls", ...) = 0
356526 ... 19 calls from ls (openat, getdents64, close...) ...
356526 write(1, "a.txt\nb.txt\n", 12) = 12
356526 exit_group(0) = ?
356524 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 356526
356524 --- SIGCHLD {si_signo=SIGCHLD, ...} ---
356524 write(2, "launch: ls exited with status 0\n", ...)
```

## wait status word

**0x0000**

```
WIFEXITED(s) = true
WEXITSTATUS(s) = 0
WIFSIGNALED(s) = false
```

# If every process is born from a `fork`, who is the first?

## Process 0

One per CPU (*idle*), built by the kernel, in kernel mode: it initializes virtual memory, creates the kernel threads ( `kthreadd` , PID 2) and process 1.

## Process 1

It switches to user mode and runs `/sbin/init`, which today is **systemd**: it starts the services and adopts the orphans.

### Real output

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
...
```

## Ending a process: `_exit` and `exit`

C

```
void _exit(int status);    /* system call */  
void exit(int status);    /* atexit + flush stdio + _exit */
```

The status (0–255) is collected by the parent with `wait`; the shell shows it in `$?`. Non-zero = error. Neither returns.

- Files are closed and pages freed (except shared ones)
- Living children go to `init` or to the nearest *subreaper*
- The parent receives `SIGCHLD` (ignored by default)
- If it was a session leader with a terminal, the others get `SIGHUP`
- Prefer `exit`; use `_exit` in the child after a failed `exec`

## Sessions and services (daemons)

- A **session** groups processes (typically, those of a terminal) with a leader
- `setsid()`: creates a new session without a controlling terminal; -1 if it is already a leader
- Inside the session, **process groups** (a shell job: `a | b`): `setpgid / getpgid`; `getsid` and `tcgetsid` query the session of a process and of a terminal
- Classic daemon: `fork`, the parent exits, the child calls `setsid`

### 2026 update

On Linux services are launched by **systemd**: the program runs in the foreground and systemd handles the session, the logs (`journalctl`), restarts and limits (cgroups). There is no need to "daemonize" any more.

# Waiting for a child: `wait` and `waitpid`

C

```
pid_t wait(int *status);           /* = waitpid(-1, status, 0) */
pid_t waitpid(pid_t pid, int *status, int options);
/* pid > 0: that child · -1: any child · 0 / < -1: a process group
   options: WNOHANG (do not block) · WUNTRACED (stopped by SIGSTOP,
   SIGTSTP, SIGTTIN, SIGTTOU) · WCONTINUED (resumed by SIGCONT) */
```

A finished child stays a **zombie** until the `wait`:

Real output

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has
called wait yet:
      PID    PPID STAT COMMAND
      356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7;
now ps finds nothing:
      PID    PPID STAT COMMAND
...
```

# How did it end? The W... macros

WIFEXITED → ended with `exit`

WEXITSTATUS

WIFSIGNALED → killed by a signal

WTERMSIG

WIFSTOPPED → stopped (with

WSTOPSIG

`WUNTRACED` )

WIFCONTINUED resumed (with

`WCONTINUED` )

C tds/td3/launch.c

```
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
if (WIFEXITED(status)) {
    fprintf(stderr, "launch: %s exited with status %d\n",
            return WEXITSTATUS(status);
}
if (WIFSIGNALED(status)) {
    fprintf(stderr, "launch: %s killed by signal %d (%s",
            signame(WTERMSIG(status)), WCOREDUMP(status),
    return 128 + WTERMSIG(status); /* sh
}
```



# Solved TD3 + AI challenge

- The 5 `buffer` variants on a terminal and on a pipe
- `fork_pid` and its three fixes; `launch` with three endings
- New: `zones`, `lazy`, `cow` and `lifecycle` (measurements)

Bash

```
make -C tds && tds/tests/run.sh td3
```



Universidad  
Indoamérica

## TAKE-AWAYS

---

# Three questions

1. Why does forking a 256 MiB process take milliseconds?
2. What does a process keep after `execve`?
3. What is a zombie and who removes it?